Proyecto de Diseño 3

Cerrojos, Flip-flops y Registros

El objetivo de esta práctica es estudiar las características de los cerrojos, flip-flops, y registros.

Parte 1

Los dispositivos reconfigurables de Altera incluyen flip-flops que están disponibles al usuario para la implementación de sus diseños. En esta práctica mostraremos como hacer uso de estos flip-flops en las partes IV y VII. Pero primero se mostrará como crear elementos de almacenamiento en un FPGA sin hacer uso de los flip-flops dedicados.

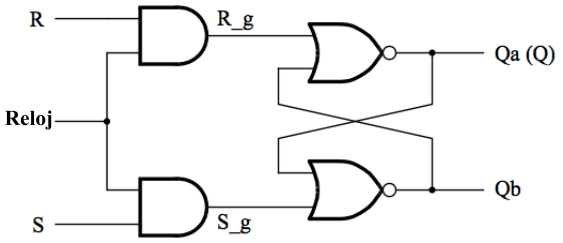


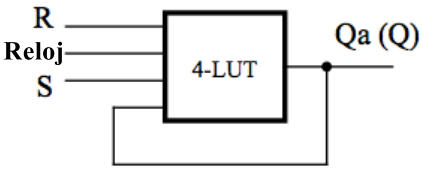
Figura 1: Estructura con compuertas de un cerrojo RS con entrada de habilitación.

```
1  ENTITY parte1 IS
2    PORT ( Reloj, R, S : IN  BIT;
3           Q           : OUT BIT);
4  END parte1;
5
6  ARCHITECTURE estructural OF parte1 IS
7    SIGNAL R_g,S_g,Qa,Qb : BIT;
8    ATTRIBUTE keep : boolean;
9    ATTRIBUTE keep of R_g,S_g,Qa,Qb : SIGNAL IS true;
10 BEGIN
11   R_g <= R AND Reloj;
12   S_g <= S AND Reloj;
13   Qa <= NOT (R_g OR Qb);
14   Qb <= NOT (S_g OR Qa);
15
16   Q <= Qa;
17
18 END estructural;
```

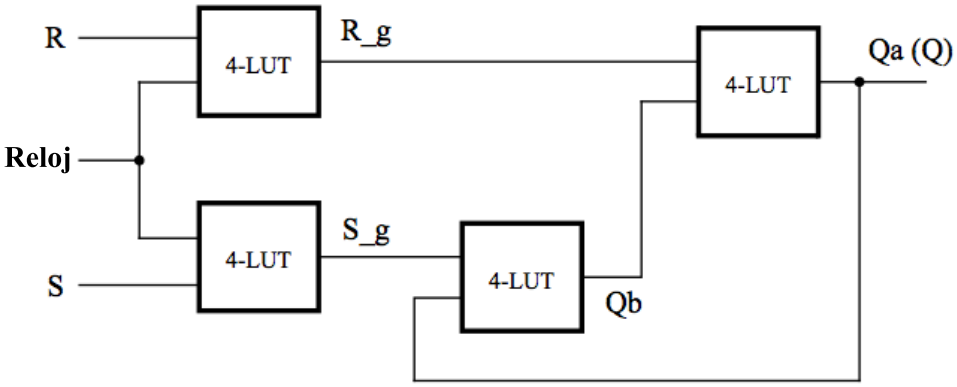
Figura 2: Descripción estructural del cerrojo RS.

La Figura 1 ejemplifica el diseño basado en compuertas de un cerrojo tipo RS. El código ejemplo en VHDL mostrado en la Figura 2 usa lógica combinatoria para describir el comportamiento del cerrojo tipo RS. Si este cerrojo se implementa en un FPGA con tablas de consulta (LUT por su abreviatura en inglés) de cuatro entradas, solo se necesitaría una sola tabla como se muestra en la Figura 3.a.

Aunque el cerrojo RS puede ser correctamente implementado usando solo una LUT de cuatro entradas (4-LUT), esta implementación no permite observar las señales internas (e.g. R_g o S_g). Para conservar



(a) Implementación del cerrojo RS con un elemento 4-LUT.



(b) Implementación del cerrojo RS usando cuatro 4-LUT.

Figura 3: Implementación del cerrojo RS de la Fig. 1.

estas señales internas en la implementación con LUTs es necesario incluir algunas *directivas del compilador* dentro de nuestro código. Dichas directivas se muestran en las lineas 11-12 del código mostrado en la Figura 2, e instruyen al compilador del Quartus II a usar elementos lógicos separados para cada una de las señales *R_g*, *S_g*, *Qa* y *Qb*. La compilación de este código produce un circuito con cuatro 4-LUTs como el mostrado en la Figura 3.b.

Cree un proyecto en el Quartus II para el circuito del cerrojo RS como a continuación se menciona:

1. Cree un nuevo proyecto en el IDE Quartus II. Seleccione el dispositivo Cyclone II EP2C35F672C6 como destino de su diseño, que es el dispositivo reconfigurable usado en la tarjeta DE2.
2. Añada en un archivo de diseño nuevo el código VHDL mostrado en la Figura 2, y asocielo a su proyecto actual.
3. Compile su proyecto. Use la herramienta *RTL Viewer* en el IDE Quartus II para examinar el circuito producido a nivel de compuertas, y use la herramienta *Technology Viewer* para verificar que el cerrojo es implementado como se muestra en la Figura 3.b.
4. Cree un archivo de formas de onda (*.vwf) que especifique las entradas y salidas del circuito. Asigne trenes de pulsos a las entradas R y S de forma que al simular su circuito usando la herramienta *Simulator*, usted sea capaz de observar las señales internas *R_g*, *S_g*, *Qa* y *Qb*. Verifique que el cerrojo funciona como se esperaba usando tanto la simulación funcional como la temporal.

Parte 2

La Figura 4 muestra el circuito para el cerrojo tipo D.
Realize los siguientes pasos:

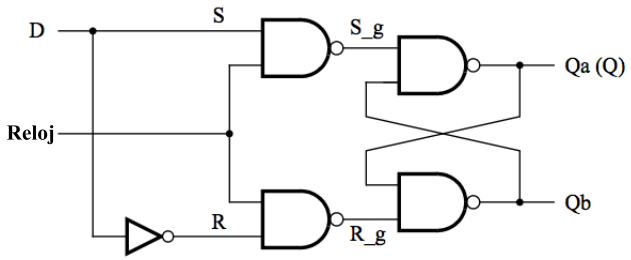


Figura 4: Estructura con compuertas de un cerrojo tipo D con entrada de habilitación.

1. Cree un proyecto nuevo en el IDE Quartus II. Escriba el programa en VHDL para el cerrojo tipo D basándose en el código mostrado en la Fig. 2. Use la directiva *keep* para asegurarse que las señales R, S_g, R_g, Q_a y Q_b son implementadas usando elementos lógicos separados.
2. Compile su código utilizando el dispositivo Cyclone II EP2C35F672C6 como destino de su diseño. Use la herramienta *Technology Viewer* para examinar el circuito implementado.
3. Verifique que el cerrojo funciona correctamente para todas las condiciones de entrada, usando la herramienta de simulación funcional. Examine el tiempo de propagación de las señales en su circuito usando la herramienta de simulación temporal.
4. Cree un nuevo proyecto en el IDE Quartus II, el cual será usado para implementar el cerrojo tipo D en la tarjeta DE2. Este proyecto deberá consistir de una entidad que contenga los puertos de entrada/salida adecuados para la tarjeta DE2. Instacie su diseño anterior en esta nueva entidad. Use el interruptor SW_0 como fuente para la entrada D del cerrojo, y use SW_1 como la entrada de reloj *Reloj*. Conecte la salida Q al LED $LEDR_0$.
5. Recompile su proyecto y programe la tarjeta DE2.
6. Pruebe que si diseño funciona correctamente.

Parte 3

La Figura 5 muestra dos cerrojos tipo D en configuración maestro/esclavo.

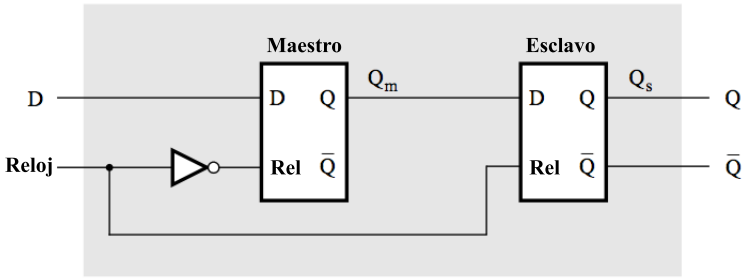


Figura 5: Implementación de un flip-flop tipo D con disparo positivo mediante un arreglo maestro/esclavo de dos cerrojos tipo D con habilitación.

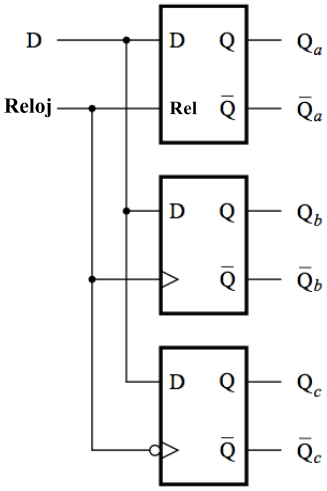
Ejecute los siguientes pasos:

1. Cree un proyecto nuevo en el IDE Quartus II. Escriba un programa en VHDL que instancie dos copias del cerrojo tipo D de su diseño anterior, y conectelas en la configuración maestro/esclavo.

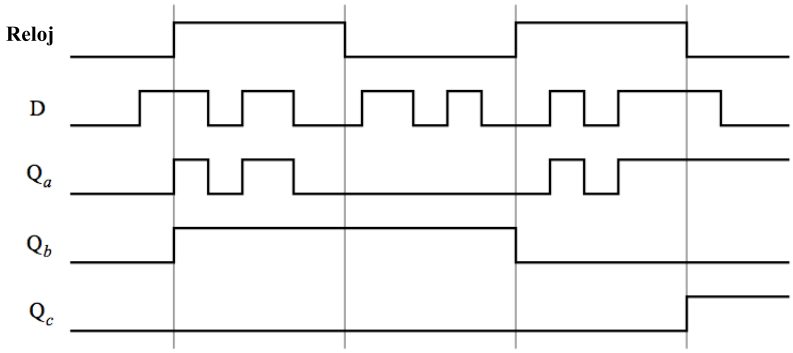
2. Incluya en su proyecto entradas y salidas tal que el interruptor SW_0 sea la entrada D del flip-flop, y use SW_1 como la entrada *Reloj*. Conecte la salida Q al LED $LEDR_0$
3. Compile su proyecto,
4. Use la herramienta Technology Viewer para examinar el circuito generado, y use la herramienta de simulación para verificar su correcta operación.
5. Programe la tarjeta DE2 con su diseño y pruebe su correcto funcionamiento.

Parte 4

La Figura 6.a muestra un circuito con tres elementos de almacenamiento distintos: un cerrojo tipo D con entrada de habilitación, un flip-flop tipo D con disparo positivo, y un flip-flop tipo D con disparo negativo.



(a) Circuito



(b) Formas de onda resultantes de la simulación

Figura 6: Circuito y formas de onda para la Parte 4.

Implemente y simule este circuito usando el IDE Quartus II siguiendo los pasos listados a continucación:

1. Cree un nuevo proyecto.
2. Escriba un programa en VHDL que instancie los tres elementos de almacenamiento. Para esta parte no es necesario que siga usando la directiva *keep* (incluida en la instrucción ATTRIBUTE) en su

código. El código mostrado en la Figura 7 describe el cerrojo tipo D con entrada de habilitación usando un estilo de programación conductual, donde no se involucra la equación booleana del circuito, solo su comportamiento. El circuito sintetizado a partir del código mencionado, se implementa usando solo un elemento LUT (abreviatura del inglés *LookUp Table*) de cuatro entradas. Use el mismo estilo de programación para describir los flip-flops restantes de la Figura 6a.

3. Compile su proyecto y sue la herramienta Technology Viewer para examinar el circuito implementado. Verifique que el cerrojo solo usa un elemento LUT y que los flip-flops no fueron implementados usando compuertas individuales sino mediante los flip-flops integrados en el FPGA.

4. Cree un archivo de formas de onda (extension *.vwf) para la simulación de su proyecto. Asigne las formas de onda mostradas en la Figura 6.b a las entradas D y *Reloj* de su proyecto y use la herramienta de simulación funcional para obtener las formas de onda correspondientes a las tres salidas. Observe el diferente comportamiento de los tres elementos.

```
1  ENTITY cerrojo_D IS
2    PORT ( D,Reloj : IN  BIT;
3           Q       : OUT BIT);
4  END cerrojo_D;
5
6  ARCHITECTURE comportamiento OF cerrojo_D IS
7  BEGIN
8    PROCESS (D,Reloj)
9    BEGIN
10     IF Reloj='1' THEN
11       Q <= D;
12     END IF;
13   END PROCESS;
14 END comportamiento;
```

Figura 7: Descripción del cerrojo tipo D usando programación conductual.

Parte 5

1. Diseñe un circuito que despliegue el valor hexadecimal de dos números binario de 16 bits, A y B, en los visualizadores de siete segmentos $HEX7-4$ y $HEX3-0$, respectivamente. Los valores para A y B serán controlados mediante los interruptores SW_{15-0}. Cuando se encienda el circuito, el estado de SW_{15-0} se asignará a A al presionar el botón KEY_1. Al presionar por segunda vez KEY_1, el estado de SW_{15-0} se asignará a B. El estado inicial de los segmentos en cada visualizador debe deser apagado. El circuito debe de contar con una entrada activa en bajo que restablesca los visualizadores a su estado incial.

2. Cree un nuevo proyecto en el IDE Quartus II para implementar su diseño en la tarjeta DE2.

3. Escriba el programa en VHDL que describa el circuito mencionado. Use el botón KEY_0 como la entrada de restablecimiento activa en bajo, y KEY_0 como entrada de reloj para su circuito. Incluya el programa en su proyecto y compilelo.

4. Asigne los pines adecuados del FPGA para conectar los interruptores y visualizadores de siete segmentos a las entradas y salidas de su diseño.

5. Recompile el proyecto y programe la tarjeta DE2 con su diseño.

6. Comprube la correcta operación de su circuito.